MuleNet: Real-Time Detection of Money Mule Accounts on IBM LinuxONE

Abstract submitted to IBM Z Datathon 2026

Theme: Real-Time AI for Critical Decisions
Team: Cryptic Vectors, Department of Artificial Intelligence and Data Science
Sona College of Technology, Salem

1. Problem Statement

In most digital payment scams the stolen money does not stay in the scammer's own account. It is quickly moved through "mule" accounts, which are bank accounts that have been rented, sold or misused, many of them belonging to students or low-income people who were offered easy money. The money usually comes in and goes out within minutes and passes through more than one account. Because of this, banks often find the mule account only after the money has already left, and the victim has very little chance of getting it back.

This problem is serious enough that the Reserve Bank Innovation Hub has developed an ML tool, MuleHunter.AI, which has been reported to be in use across 31 banks [3]. Our aim is to show how this kind of decision can be made inside the transaction itself, in real time, before the next transfer goes out.

2. Proposed Solution

MuleNet is a real-time scoring system that runs on IBM LinuxONE. Every incoming transaction updates a behaviour profile of the receiving account. A machine learning model then estimates how likely it is that the account is being used as a mule. Depending on the score, the system allows the transaction, sends a warning, or holds the outgoing transfer and alerts an investigator. Each decision comes with a short explanation of the main reasons behind it.

3. System Architecture

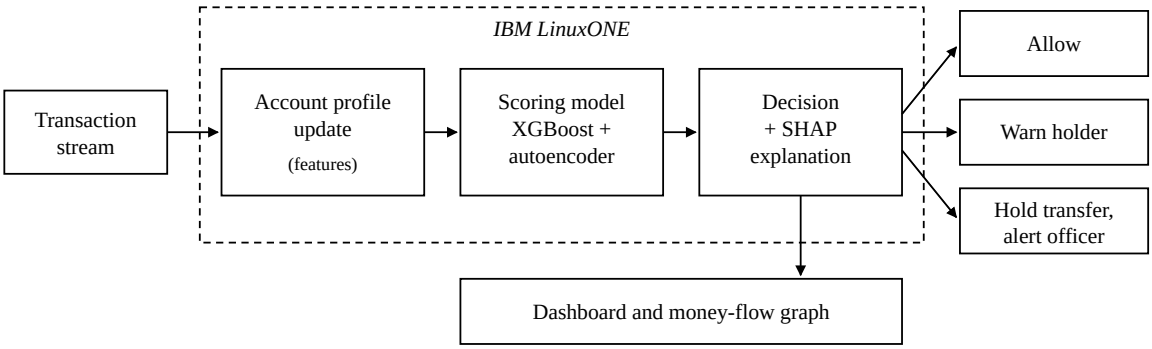


Figure 1. Overall flow of MuleNet

4. Methodology

4.1 Dataset

We use the IBM Transactions for Anti-Money Laundering (AML) dataset [1], [2]. It contains synthetic bank transactions generated by IBM researchers, where each transaction is labelled as laundering or normal. The data is sorted by time. The earlier part (about 80%) is used for training and the later part (about 20%) is replayed one row at a time to act as a live transaction stream.

4.2 Feature engineering

Since mule behaviour is visible at the account level, we build features per account: number of unique senders (fan-in) and receivers (fan-out), total money in and out, pass-through ratio (money out divided by money in), average time money stays in the account before leaving, and transaction velocity. These are updated after every transaction.

4.3 Models

XGBoost is the main classifier, with Random Forest as a baseline. Class weights are used because laundering cases are rare. An autoencoder trained only on normal accounts is added to flag unusual behaviour that the labelled data may not cover (Isolation Forest is kept as a fallback). The final risk is a weighted combination of both scores. SHAP is used to list the top reasons for each alert.

4.4 Decision and actions

The risk score is mapped to three levels: low (allow), medium (warn the account holder, who may not know their account is being misused) and high (hold the outgoing transfer and alert an investigator). For high-risk accounts, a simple graph search follows the money one or two steps forward so that the next accounts in the chain can also be flagged.

5. What Is New in Our Approach

1. The decision is taken while the transaction is happening, not in a later review.
2. The focus is on the receiving account's behaviour, not only on single payments.
3. Alerts explain themselves and name the likely laundering pattern (fan-in, fan-out or cycle).
4. The next accounts in a mule chain are flagged early.
5. Account holders who are unknowingly being used get a warning instead of only a freeze.

6. Why IBM Z / LinuxONE

Core banking data is already stored on IBM Z in many banks, so scoring on the same platform avoids sending sensitive data to another system and keeps the delay low. In an IBM client example presented in the datathon's "AI on IBM" session, moving a fraud model onto IBM Z reduced scoring time from about 200 ms to about 1 ms. IBM Z's built-in encryption and its high availability also suit a system that must check every transaction without downtime.

7. Evaluation Plan

Because the data is highly imbalanced, we will report precision, recall, F1 score and PR-AUC instead of accuracy. We will also measure scoring latency in milliseconds, how many transactions it takes to detect a mule account, and the amount of money that would have been stopped before leaving.

8. Technology Stack

Part	**Tools**
Data and models	Python, pandas, NumPy, scikit-learn, XGBoost, TensorFlow/Keras, SHAP, NetworkX
Real-time service	FastAPI, Uvicorn, SQLite, Python stream simulator
Dashboard	Streamlit, Plotly, PyVis
Platform	IBM LinuxONE Community Cloud, GitHub

9. Future Work

Graph neural networks that learn directly from the transaction network, sharing of risk signals between banks without sharing raw data, and running the model on the IBM Telum on-chip AI accelerator.

10. Team Members

S. Harsha Vardhaan (Team Lead), Benadict Infant A, Ameerkhan S, Kanisha M, B R Devamayookha, Nivethitha A G

References

1. IBM Transactions for Anti Money Laundering (AML), Kaggle dataset. kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml
2. E. Altman et al., "Realistic Synthetic Financial Transactions for Anti-Money Laundering Models," arXiv:2306.16424, 2023.
3. "What is MuleHunter.AI? RBI's latest tool against financial fraud explained," Business Standard, Dec. 2024.